

1. Feladat

Egyszerű példa feladatok az elemi programozási tételekre. Adott egy 15 elemű tömb mely 0 és 100 között tartalmazza 15 tanuló elért vizsgapontszámait. Alkalmazzuk az elemi programozási tételeket!

1. Számítsuk ki a csoport átlag pontszámát!
2. Döntsük el volt-e maximum pontszám (100 pont)!
3. Tudjuk, hogy volt 57 pontos dolgozat. Adjuk meg annak a tanulónak a sorszámát, aki 57 pontot kapott! Ha több van elég az első! (az első elem sorszáma 0 de a tanulók számozása 1-el kezdődik)
4. Kérjünk be egy pontszámot és ha volt ilyen a tömbben adjuk meg a hozzá tartozó tanuló sorszámát!
5. Mennyien feleltek meg a vizsgán? (A pontszám > 50)
6. Válogassuk ki a legalább 4-est elérő tanulók (>75) sorszámát és gyűjtsük egy új tömbbe őket, majd írassuk ki az új tömb elemeit!
7. Hányadik tanuló érte el a legkevesebb pontot és az mennyi volt?

```
#include <iostream>
using namespace std;
```

```
int main( )
{
    int tomb[15]={1,34,56,4,57,45,67,6,43,32,33,99,33,100,1};
```

```
// Összegzés - számítsuk ki a csoport átlag pontszámát
int i,sum=0;
for (i = 0; i < 15; i++)
{
    sum = sum + tomb[i];
}
cout<<"A csoport atlagpontszama: "<<(float)sum/i<<endl;
```

```
// Eldöntés - döntsük el volt-e MAX pontszám (100 pont)
i=0;
while (i < 15 && tomb[i] != 100)
{
    i++;
}
if(i<15) cout<<"Volt 100 pontos dolgozat."<<endl;
else cout<<"Nem volt 100 pontos dolgozat."<<endl;
```

```
// Kiválasztás - Tudjuk, hogy volt 57 pont adjuk meg a hozzá tartozó tanuló sorszámát!
//(a tanuló sorszáma a tömbbeli index+1 elem sorszáma)
i=0;
while (tomb[i] != 57)
{
    i++;
}

cout<<"A keresett sorszam a "<<i+1<<endl;
```

```
//Lineáris keresés kérjünk be egy pontszámot és ha volt ilyen a tömbben adjuk meg a hozzá tartozó
//tanuló sorszámát!
int szam;
cout<<"Adja meg a keresett szamot!";
cin>>szam;
i=0;
while (i < 15 && tomb[i] != szam)
```

```

        {
            i++;
        }
    if(i<15)cout<<" A keresett tanulo sorszama: "<<i+1<<endl;
    else cout<<" Nincs a keresett szam a tombben!"<<endl;

// Megszámlálás- Hányan feleltek meg a vizsgán? (A pontszám > 50)
int db50=0;
    for (i = 0; i < 15; i++)
    {
        if (tomb[i] > 50) db50++;
    }
    cout<<"A megfelelték szama: "<<db50<<endl;

// Kiválogatás- válogassuk ki a legalább 4-est elérők (>75) indexét
// és gyűjtjük egy új tömbbe őket, majd írassuk ki az új tömb elemeit!
int cdb=0,c[15];
    for (i = 0; i < 15; i++)
    {
        if (tomb[i] > 75)
        {
            c[cdb++] = i + 1;
        }
    }
    cout<<"A 4-est elerok sorszamai: " <<endl;
    for (i = 0; i < cdb; i++)
    {
        cout << c[i] << " ";
    }
    cout<<endl;
// Minimum/Maximum kiválasztás - hányadik tanuló érte el a legkevesebb pontot
// és az mennyi volt ?
int min=0;
    for (int i = 1; i < 15; i++)
    {
        if (tomb[i] < tomb[min])
        {
            min = i;
        }
    }
    cout<<"A leggyengébb tanulo eredménye "<<tomb[min]<<" sorszama "<<min+1<<endl; // a tanuló sorszáma
a kérdés
    return 0;
}

```

2. Feladat

A kosarlabda.txt állományban tároljuk egy kosarlabda csapat egyik mérkőzésének statisztikáját. Tárolják a játékos nevét, dobott kosarait és a mérkőzés alatt a pályán töltött időt percben. (Csak ékezetmentes betűket írtak)

Pl. Kovacs 13 40
Fekete 2 23

.....

Minden játékos adata új sorban van és a nevek, kosarak, eltöltött idő szóközzel elválasztva. Tudjuk, hogy legfeljebb 14 játékos adatai vannak az állományban (nem biztos, hogy van 14)

1. Olvassa be a fájlt egy olyan adatszerkezetbe, amit az alábbi kérdésekhez fel tud használni! (struktúra)
2. Mennyi játékos volt jelen a mérkőzésen?
3. Mennyi kosarat dobott a csapat átlagosan a mérkőzésen?
4. Döntse el volt-e olyan játékos, akinek 30 pont felett sikerült dobnia a mérkőzésen (igen/nem)!
5. Kérjen be egy nevet billentyűzetről és döntse el, hogy játszott-e, és ha igen, akkor hány pontot szerzett?
6. Írja ki a képernyőre annak a játékosnak a nevét, aki a legtöbb időt töltötte a pályán.
7. Válogassa ki azoknak a játékosoknak a nevét egy új tömbbe, akik 20 percnél többet töltöttek a pályán!

Az egyes feladatokat függvényekkel oldja meg!

```
#include<iostream>
#include<string>
#include<iomanip>
#include<fstream>
using namespace std;
struct kosar
{
    string nev;
    int pont;
    int ido;
};
int Adatbeolvasas(kosar* tmb );
double Pont_atlag(kosar* tmb, int db);
bool Harminc_pont_felett(kosar* tmb, int db);
int Jatszott(kosar* tmb, int db, string jatekos);
string Legtobb(kosar* tmb, int db);
int Huszperc(kosar* tmb, int db,string *nevek);
int main()
{
    //1, Csapat adatainak beolvasása
    kosar csapat[14];
    int letszam = Adatbeolvasas(csapat);

    //2,Hány játékos volt jelen a mérkőzésen?
    cout << "A merkozesen "<< letszam <<" jatekos vett részt" << endl;

    //3, A csapat által dobott pontok átlaga
    cout << "A csapat által dobott pontok atlaga: " << Pont_atlag(csapat, letszam) << endl;

    //4, Volt-e harminc pont felett dobó
    if (Harminc_pont_felett(csapat, letszam)) cout << "VOLT 30 pontnál többet elero jatekos" << endl;
    else cout << "NEM VOLT 30 pontnál többet elero jatekos" << endl;

    //5, Adjon meg egy nevet
```

```

        cout << "Adjön meg egy nevet: " << endl;
        string Nev;
        cin >> Nev;
        if (Jatszott(csapat, letszam, Nev)>=0) cout << "A jatekos jatszott es " << Jatszott(csapat, letszam, Nev)
<< " pontot dobott" << endl;
        else cout << "A jatekos nem jatszott " << endl;

//6, A legtöbb időt pályán töltő játékos
cout << "A legtöbb idot palyan tolto jatekos neve: " << Legtobb(csapat, letszam) << endl;;

//7, A 20 percnél többet pályán töltők nevei - létrehozunk egy string tömböt a neveknek és azt is átadjuk
// a függvénynek, a függvény ezen játékosok számát adja vissza a neveket pedig betölti a string tömbbe
string jatekosok[14];
cout << "Husz percnel tobbet jatszott :" << endl;
for (int i = 0; i < Huszperc(csapat, letszam, jatekosok); i++)
{
    cout << jatekosok[i] << endl;
}
}
int Adatbeolvasas(kosar* tmb )
{
    ifstream be("kosarlabda.txt");
    if (be.fail()) { cout << "Hiba a file beolvasasnal."; system("pause"); exit(1); }
    int i = 0;
    cout.setf(ios::left);
    while (!be.eof())
    {
        be >> tmb[i].nev >> tmb[i].pont >> tmb[i].ido;
        cout << setw(20) << tmb[i].nev << "\t" << tmb[i].pont << "\t" << tmb[i].ido << endl;
        i++;
    }
    be.close();
    return i;
}
double Pont_atlag(kosar* tmb, int db)
{
    float atlag = 0;
    for (int i = 0; i < db; i++)
    {
        atlag = atlag + tmb[i].pont;
    }
    return atlag/db;
}
bool Harminc_pont_felett(kosar* tmb, int db)
{
    int i = 0;
    while (i < db && tmb[i].pont <= 30)
    {
        i++;
    }
    if (i < db) return true;
    else return false;
}

int Jatszott(kosar* tmb, int db, string jatekos)
{
    int i = 0;
    while (i < db && tmb[i].nev != jatekos)
    {
        i++;
    }
}

```

```

    }
    if (i < db) return tmb[i].pont;
    else return -1;
}

string Legtobb(kosar* tmb, int db)
{
    int max = 0;
    for (int i = 1; i < db; i++)
    {
        if (tmb[i].ido > tmb[max].ido) max = i;
    }
    return tmb[max].nev;
}

int Huszperc(kosar* tmb, int db, string* nevek)
{
    int szam = 0;
    for (int i = 0; i < db; i++)
    {
        if (tmb[i].ido > 20)
        {
            nevek[szam] = tmb[i].nev;
            szam++;
        }
    }
    return szam;
}

```

3. Feladat

Egy utazó ügynök az alábbi 10 városban járt januárban: Vac, Pecs, Szeged, Tata, Vac, Gyor, Pecs, Tata, Vac, Eger

Válogassuk ki a városokat egy új tömbbe úgy, hogy mindegyik név csak egyszer szerepeljen benne!

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
// Egy utazó ügynök az alábbi 10 városban járt januárban : Vac, Pecs,Szeged, Tata, Vac, Gyor, Pecs, Tata,
// Vac, Eger
// Válogassuk ki a városokat egy új tömbbe úgy hogy mindegyik név csak egyszer szerepeljen benne!
string varosok[10] = { "Vac", "Pecs", "Szeged", "Tata", "Vac", "Gyor", "Pecs", "Tata", "Vac", "Eger" };
string uj[10];
int ujdb = 0; //az uj tömb darabszáma- jelenleg 0 üres a tömb
for (int i = 0; i < 10; i++) // vegyük egyenként a városokat
{
    int j = 0;
    while (j < ujdb && varosok[i] != uj[j]) // döntsük el, hogy az adott város benne van-e már az új tömbben
    {
        j++;
    }
    if (j == ujdb) //ha úgy szálltunk ki a ciklusból, hogy nem volt az adott város az uj tömbben
    {
        uj[ujdb] = varosok[i];
        ujdb++; // növeljük az uj tömbben lévő városok számát
    }
}
for (int i = 0; i < ujdb; i++)
{
    cout << uj[i] << " ";
}
return 0;
}
```